

InFP: A 17.97 TFLOPS/W Reconfigurable SRAM Based Computing-in-Memory Macro for BF16 MAC Operations

Jing Zhang^{1,2}, Tianchu Dong¹, Shaoxuan Li¹, Zhengxi Yan¹, and Shanshi Huang^{1†}

¹Microelectronics Thrust, The Hong Kong University of Science and Technology (Guangzhou), China

²Guangdong-Macao Joint Laboratory for Modular Chip Design and Testing

[†]Email: shanshihuang@hkust-gz.edu.cn

Abstract—As the demand for high-precision training and inference of deep neural networks (DNNs) continues to grow, floating-point (FP) compute-in-memory (CIM) architecture has emerged as an essential solution. However, in contrast to extensively studied integer (INT) CIM, the efficiency of FP-CIM still requires further enhancement. In this paper, we present an energy-efficient and area-efficient digital SRAM-based FP-CIM macro with reconfigurable peripheral logic. Specifically, we design an interleaved SRAM bank with specific weight mapping, which improves SRAM utilization and enables write-compute pipeline scheduling to maximize hardware usage. Additionally, to improve energy efficiency and reduce area requirements, we propose a reconfigurable peripheral logic equipped with a shared addition unit, conducting exponent and mantissa addition at different stages of the FP MAC operation. A 16kb SRAM CIM macro with the proposed techniques is designed in TSMC 45nm CMOS technology and occupies an area of 0.081 mm². The simulation results show that our work achieves a peak energy efficiency of 17.97 TFLOPS/W and an area efficiency of 0.395 TFLOPS/mm² with brain-float 16 (BF16) representation.

Index Terms—computing-in-memory, floating-point, multiply-accumulate, deep neural networks

I. INTRODUCTION

With the rapid development of artificial intelligence (AI), deep neural networks (DNNs) have attained significant success in applications such as image recognition, natural language processing, and other specific research areas [1]–[4]. This progress pushes DNN models toward greater depth and complexity, which greatly increases the number of multiply-accumulate (MAC) operations, creating substantial computational overhead for both inference and training. As a result, there is an urgent need for specialized hardware accelerators to efficiently handle these computations.

Among various hardware solutions, Computing-in-Memory (CIM) has become a promising approach to improve energy efficiency by mitigating the data movement bottleneck. DNN accelerators primarily leverage two types of CIM technologies: analog CIM and digital CIM. Analog CIM processors typically store all parameters on-chip, thereby eliminating off-chip DRAM access and significantly improving performance [5]–[7]. However, they face challenges such as limited scalability and circuit non-idealities. In contrast, digital CIM, which integrates digital computing logic directly into SRAM arrays, has

recently gained increasing attention for its design flexibility, high accuracy and efficient data reloading capabilities [8], [9].

Early digital computing-in-memory (DCIM) designs [10]–[13] primarily supported fixed-point arithmetic, constrained by the physical nature of crossbar architectures. While demonstrating good efficiency, this limitation hinders their ability to execute inference on more complex network models or to support DNN training tasks effectively. For example, the inference phase for large language models (LLMs) often employs a 4-bit integer and 16-bit floating-point (INT4 × FP16) format to maintain accuracy [14], while the training phase relies on high-precision floating-point MAC operations, such as brain-float 16 (BF16) or FP32 [15]. Thanks to the inherent flexibility of DCIM, researchers have begun exploring FP-CIM processors. Mai et al. [16] proposed a design incorporating separate exponent and mantissa CIM arrays, achieving high throughput at the cost of area efficiency. Li et al. [17] introduced an energy-efficient DCIM architecture supporting approximate logarithmic FP MAC, improving energy efficiency but sacrificing computational parallelism and introducing inevitable accuracy loss. Therefore, developing a FP-CIM architecture that simultaneously maintains high accuracy, energy efficiency, and area efficiency is essential to advance high-precision DNN training and inference.

In this paper, we propose an energy-efficient and area-efficient digital SRAM-based FP-CIM macro with reconfigurable peripheral logic. We focus on the BF16 format as it balances between hardware overhead and precision requirement of DNN. The main contributions of this work are as follows:

- We propose the reconfigurable InFP macro, supporting mantissa/exponent modes and adopting global alignment approach to solve the large overhead and long critical path of conventional FP MAC hardware.
- We design an interleaved SRAM bank with a bank-shared multiplier, which improves SRAM utilization, and enables write-compute pipeline scheduling to maximize hardware resource usage.
- We implement a reconfigurable peripheral logic block to realize hardware sharing of different work modes, reducing area overhead without sacrificing computation precision.

[†]Corresponding author

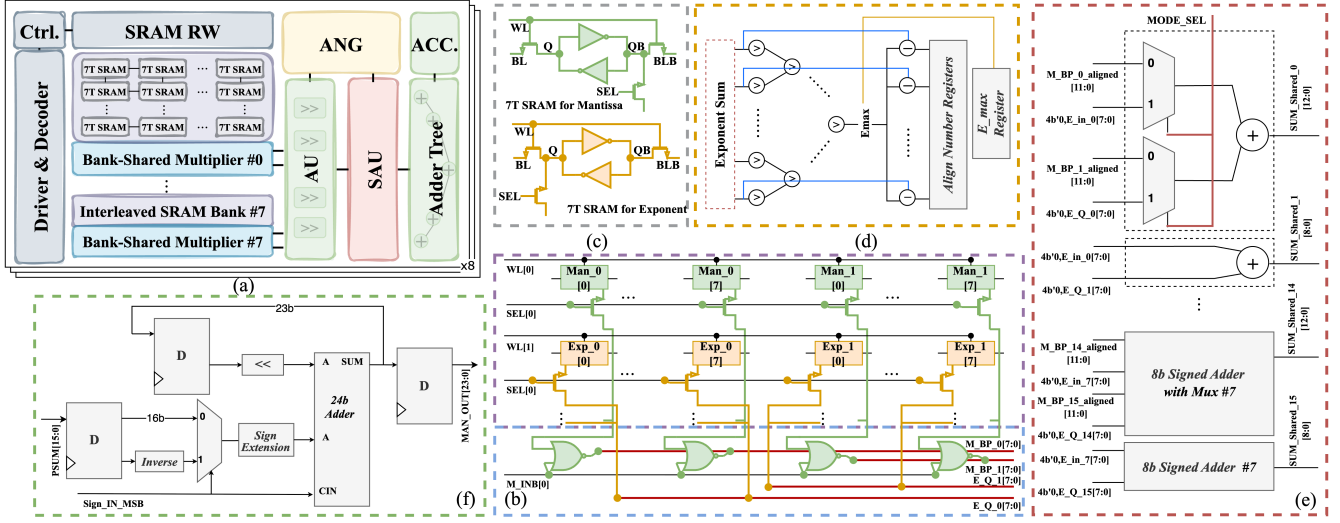


Fig. 1. (a) The overall architecture of the proposed InFP CIM macro. (b) Proposed interleaved SRAM bank and weight mapping. (c) Customized 7T SRAM cell with a selector. (d) The logic flow of ANG. (e) The detailed structure of SAU. (f) Schematic view of the accumulator.

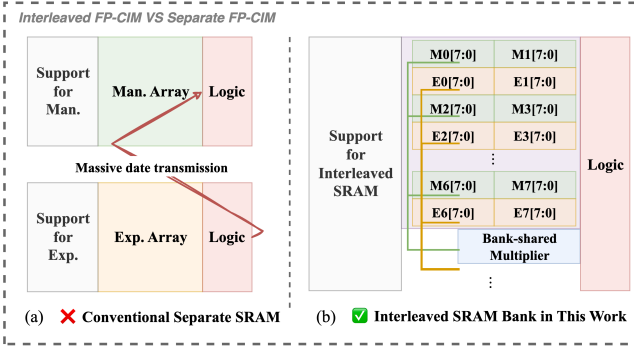


Fig. 2. Comparison between (a) conventional separate SRAM and (b) proposed interleaved SRAM bank.

II. PROPOSED INFP MACRO

A. Overall Architecture

Fig. 1 (a) presents the overall architecture of the proposed 16kb InFP macro, including eight MAC arrays, each of which mainly consists of a 2kb SRAM block, a logic block and a control block. The SRAM block is divided into eight interleaved SRAM banks with shared read-and-write (SRAM RW) units. Each bank is attached to one NOR-based multiplier, which delivers the product to the logic block. The logic block is structured with an alignment unit (AU), a shared addition unit (SAU), an align number generator (ANG), an adder tree and an accumulator (ACC). The control block includes the local control unit, drivers and decoders. The computation of the InFP macro can be configured to two modes: **mantissa mode** and **exponent mode**.

B. Interleaved SRAM Bank and Weight Mapping

Fig. 1 (b) shows the detailed circuit structure of the interleaved SRAM bank. In this design, floating-point weights, which consist of the exponent and the mantissa, are mapped to two successive rows. Specifically, the mantissa is mapped to

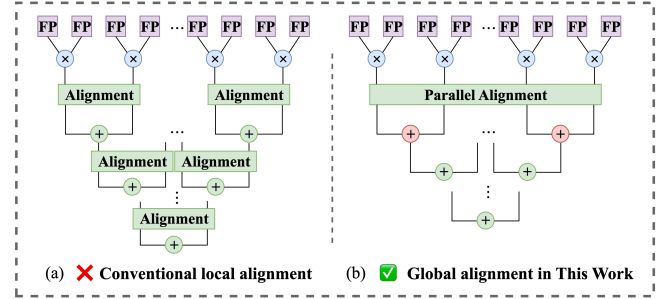


Fig. 3. (a) Conventional local alignment approach in FP MAC operation. (b) Parallel global alignment approach in this work.

the top row, and its corresponding exponent is mapped to the bottom row. Due to the BF16 format focused in this work, the bit widths of the mantissa and exponent rows are aligned. The mantissa row is connected to a shared multiplier via a selector (Fig. 1 (c), left), with the multiplier's other input being driven by the M_INB line. Similarly, the exponent row is connected to a shared bus through another selector (Fig. 1 (c), right), which then delivers the exponent value to the logic block for subsequent processing. This interleaved approach enables global alignment and accumulation, which will be discussed in the following chapter. Compared to the conventional method of mapping these components to separate arrays (Fig. 2) [16], it eliminates redundant peripheral circuits, mitigates inter-array data transmission, and increases the proportion of utilized bitcell area in the total SRAM module.

C. Logic Block and Workflow

Basically, the proposed macro employs a post-multiplication aligned accumulation for FP MAC. More precisely, conventional FP MAC applies local alignment for each addition (Fig. 3 (a)), while is more accurate, it introduces a large area overhead and a long critical path for parallel MAC. In contrast, previous work has shown that a global alignment

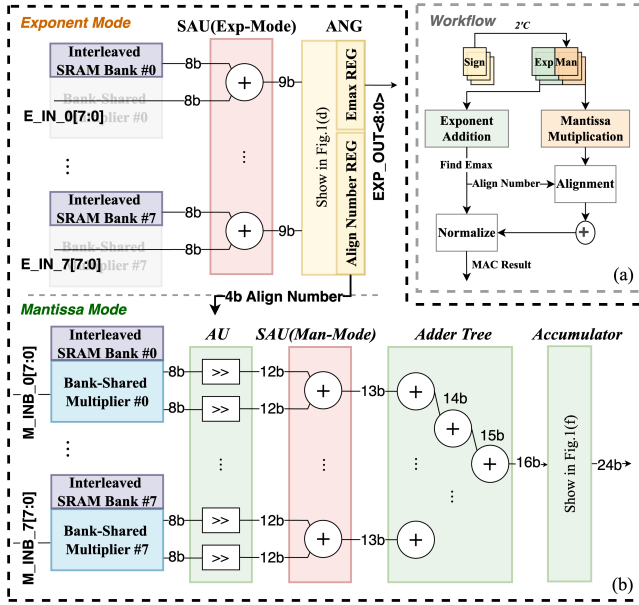


Fig. 4. (a) The whole workflow of the InFP macro. (b) The dataflow and circuit configuration of two modes.

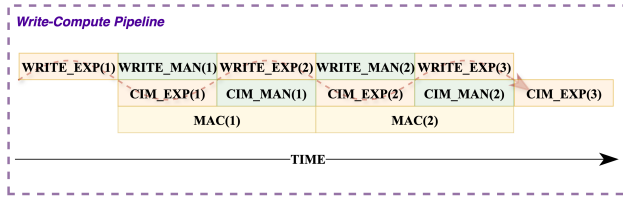


Fig. 5. The timeline of write-compute pipeline schedule.

before accumulation in a small block will not introduce accuracy degradation for FP MAC [18], which leads to a lower area overhead and a shorter critical path (Fig. 3 (a)). To implement the post-multiplication aligned accumulation, the proposed Macro will be first configured to the exponent mode, computing the align number for global alignment. Next, the macro will be switched to the mantissa mode for multiplication and aligned accumulation.

The whole workflow of the InFP macro is shown in Fig. 4 (a), with the detailed dataflow and circuit configuration described in 4 (b), dividing into an exponent mode and a mantissa mode. The operation of the Macro starts with the **exponent mode**. The specific SRAM rows storing exponents are selected, and the exponent values of the weights are output to the SAU. As shown in Fig. 1 (e), the SAU comprises sixteen 8-bit signed adders arranged in an alternating pattern of two variants: a standard adder and an adder with two MUX at its input. The adders with MUX are shared in two modes while the standard adders are only activated in the exponent mode. The SAU adds the exponents of the weights to those of the corresponding input activations, which are supplied externally to the array, as required by the FP multiplication. The 9-bit outputs of SAU are then sent to the ANG. The detailed structure of ANG is shown in 1 (e). A 4-stage comparison tree

is used to find the maximum value E_{max} among the summed exponents. Then E_{max} is used to perform the operation $(E_{max} - E_{sum})$ in the subtractor for each channel. Finally, the results of the subtractors are stored in alignment number registers and will be delivered to AU when the mode changes later. E_{max} is also stored as the final exponent of the MAC result.

Next, the macro switches to the **mantissa mode**. The specified SRAM rows for the mantissa are selected, feeding the weights to the bank-shared multiplier. Meanwhile, the mantissa of the inputs are fed into the bank-shared multiplier with an MSB-first bit-serial approach. The multipliers generate 8-bit mantissa products via bit-wise multiplication. After this, these bit-wise products of 1-bit inputs and 8-bit precision weights are sent to the AU, which mainly consists of signed shifters. Simultaneously, the 4-bit align numbers from the ANG registers are fed into the AU, specifying the bit shift amount for alignment of the mantissa products. In AU, the 8-bit products are firstly expanded to 12 bits to maintain the calculating accuracy and then right-shifted based on the align number. Next, the 12-bit aligned products are fed into the SAU, operating in mantissa mode, which works as the first stage of a conventional adder tree. The output of the SAU is then passed to the rest stages of the adder tree for channel-wise accumulation. Finally, this 16-bit partial sum is fed into the accumulator. As shown in Fig. 1 (f), it accumulates the output of adder tree in different clock cycles, which is the sum of the products of 1-bit input and multi-bit weight in all channels. As the input is bit-serial, our accumulator takes 8 cycles to complete the input-precision-wise accumulation, with the sign processing in the first cycle.

D. Write-Compute Pipeline

To further enhance the efficiency of the macro, the two modes, in conjunction with the interleaved SRAM mapping, can be facilitated by a pipeline schedule that overlaps the writing phase (updating weight mantissas or exponents in the SRAM bank) with the computing phase (performing mantissa/exponent-related arithmetic operations in the logic unit). As illustrated in Fig. 5, specifically, after the exponents are written to their corresponding rows in the SRAM bank, the EXP mode calculation can be activated in parallel with the writing of the mantissas. Subsequently, while the mantissa calculation is ongoing, the exponents for the next set of weights can be written. This overlap eliminates the idle time between write and compute tasks, ensures the continuous utilization of both the SRAM and logic components, and thereby substantially improves the overall computational throughput of the InFP macro. This enhancement is particularly beneficial when the weight reuse rate is low, such as during inference with a small batch size in a fully connected (FC) layer, or the decode stage of the LLM.

III. EVALUATION

The design is implemented and simulated using the TSMC 45nm CMOS process. The pre-layout simulation is used for

TABLE I
PERFORMANCE COMPARISON WITH PREVIOUS WORKS

	VLSI'21 [19]	ESSCIRC'22 [20]	² ISCAS'23 [16]	TCAS'23 [17]	ISCAS'24 [21]	JSSC'24 [22]	JSSC'24 [23]	This Work
Technology	28nm	28nm	40nm	40nm	28nm	28nm	28nm	45nm
Technology	Digital	Digital	Digital	Digital	Digital	Digital	Digital	Digital
Precision	BF16	FP8-FP32	BF16,FP32	BF16	FP16	BF16	FP16	INT8,BF16
Supply Voltage (V)	0.68-1.1	0.47-0.9	0.7-1.1	0.9-1.2	0.9	0.6-0.9	0.5-1.3	0.55-1.1
Memory Array Size	396kb	16kb	12kb	128kb	24kb	64kb	576kb	16kb
Area (mm ²)	5.832(sys.)	0.0439	0.48	0.41	0.033	0.146	1.53	0.081
Frequency (MHz)	250	53-403	294	625	200	-	12.5-300	200-1250
Normalized Peak Performance (GOPS/kb or GFLOPS/kb)	1.67	0.156-0.013	4.7(BF16) 1.3(FP32)	0.29	0.4	1.4	0.196	2.5(INT8) 2(BF16)
Energy Efficiency (TOPS/W or TFLOPS/W)	¹ 1.43	1.644-0.099 (FP8-FP32)	1.97(BF16) 0.62(FP32)	³ 1.94	1.97	⁴ 14.04	⁵ 21.4	⁶ 27.56(INT8) ⁶ 17.97(BF16)
Area Efficiency (TOPS/mm ² or TFLOPS/mm ²)	-	0.058-0.0048 (FP8-FP32)	0.118(BF16) 0.034(FP32)	0.091	0.290	0.615	0.113	0.494(INT8) 0.395(BF16)

¹Energy eff. at 0.76V and 40 MHz with 0% activation sparsity. ²Peak performance at 1.1V, 294 MHz. Energy efficiency at 0.7V, 53 MHz.

³at 0.9V, 625 MHz with 12.5% toggle rate. ⁴at 0.9V with 0% weight sparsity, 50% feature sparsity and 30% feature toggle rate.

⁵at 0.9V, 140 MHz with 50% input sparsity in element-level. ⁶at 0.55V with 50% input sparsity and 50% weight sparsity.

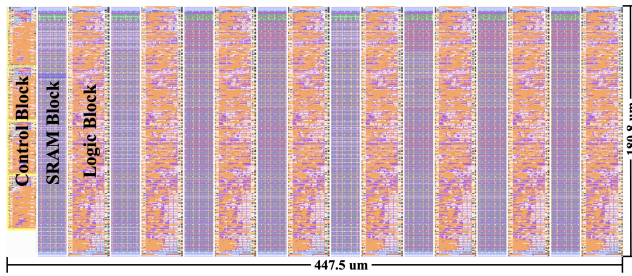


Fig. 6. Layout of proposed InFP macro.

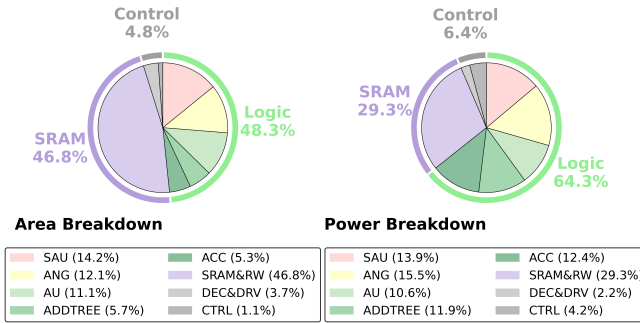


Fig. 7. Area breakdown and power breakdown for BF16 MAC operation. (SRAM&RW consists of the interleaved SRAM banks, bank-shared multipliers and SRAM read-and-write unit. ADDTREE means adder tree. DEC&DRV means decoders and drivers.)

function verification and performance evaluation while the layout is used for area estimation.

Table I presents a performance comparison with previous works. The proposed InFP macro achieves remarkable performance in terms of normalized peak performance, energy efficiency, and area efficiency, while offering the flexibility to support both INT8 and BF16 MAC operations within a single architecture. INT8 is implemented by only activating the Mantissa mode. For normalized peak performance, the proposed work attains 2.5 GOPS/kb for INT8 and 2 GFLOPS/kb for BF16. Compared to state-of-the-art (SOTA) works ([17], [22], [23]), the proposed macro achieves improvements of 5 $\times$, 1.43 $\times$ and 10.2 $\times$. In terms of energy efficiency, this macro reaches 27.56 TOPS/W for INT8 and 17.97 TFLOPS/W for

BF16. Compared with SOTA works with similar structure ([16], [17]), the proposed design exhibits energy savings of 9.26 $\times$, benefiting from optimized SRAM design and shared logic strategies. Moreover, the proposed macro achieves area efficiency values of 0.494 TOPS/mm² for INT8 and 0.395 TFLOPS/mm² for BF16. Compared with designs with conventional memory techniques (equipped with separate mantissa arrays and exponent arrays [16], [23]), our work achieves 3.35 $\times$ and 3.50 $\times$. Note that our comparison focuses on digital CIM designs that support similar data formats to our macro to ensure a fair evaluation.

The layout is shown in Fig. 6, where the InFP macro area is 0.081 mm². Fig. 7 presents the area and power breakdown of the InFP macro operating in BF16 MAC mode. In terms of area breakdown, the SRAM part accounts for 46.8% of the total area, which constitutes a memory proportion well-suited for CIM macros. Logic-related modules, including the SAU, ANG, AU, adder tree, and accumulator, contribute to a total of 48.3% of the total area. Meanwhile, the control block occupies a mere 4.8% of the total area. For power breakdown, as the primary active modules during the computation phase are logic-related, the logic part accounts for the highest proportion of power consumption, reaching 64.3%.

IV. CONCLUSION

In this work, we present the InFP macro, an energy-efficient and area-efficient digital SRAM-based FP-CIM macro with reconfigurable peripheral logic. The 16kb SRAM CIM macro with the proposed techniques is implemented in TSMC 45nm CMOS technology and occupies an area of 0.081 mm². The simulation results show that our work achieves high energy efficiency of 17.97 TFLOPS/W and area efficiency of 0.395 TFLOPS/mm² with BF16 representation. This work can provide an effective solution for high-precision DNN training and inference acceleration hardware.

V. ACKNOWLEDGMENT

This work is supported in part by the Guangdong Science and Technology Department (No.2025B1212150003) and GDIC.

REFERENCES

- [1] A. Goel, C. Tung, Y.-H. Lu, and G. K. Thiruvathukal, "A survey of methods for low-power deep learning and computer vision," in *2020 IEEE 6th World Forum on Internet of Things (WF-IoT)*. IEEE, 2020, pp. 1–6.
- [2] S. Grigorescu, B. Trasnea, T. Cocias, and G. Macesanu, "A survey of deep learning techniques for autonomous driving," *Journal of field robotics*, vol. 37, no. 3, pp. 362–386, 2020.
- [3] D. W. Otter, J. R. Medina, and J. K. Kalita, "A survey of the usages of deep learning for natural language processing," *IEEE transactions on neural networks and learning systems*, vol. 32, no. 2, pp. 604–624, 2020.
- [4] P. Khan, M. F. Kader, S. R. Islam, A. B. Rahman, M. S. Kamal, M. U. Toha, and K.-S. Kwak, "Machine learning and deep learning approaches for brain disease diagnosis: principles and recent advances," *Ieee Access*, vol. 9, pp. 37 622–37 655, 2021.
- [5] Q. Liu, B. Gao, P. Yao, D. Wu, J. Chen, Y. Pang, W. Zhang, Y. Liao, C.-X. Xue, W.-H. Chen *et al.*, "33.2 a fully integrated analog rram based 78.4 tops/w compute-in-memory chip with fully parallel mac computing," in *2020 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2020, pp. 500–502.
- [6] J.-M. Hung, Y.-H. Huang, S.-P. Huang, F.-C. Chang, T.-H. Wen, C.-I. Su, W.-S. Khwa, C.-C. Lo, R.-S. Liu, C.-C. Hsieh *et al.*, "An 8-mb dc-current-free binary-to-8b precision rram nonvolatile computing-in-memory macro using time-space-readout with 1286.4–21.6 tops/w for edge-ai devices," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 1–3.
- [7] P.-C. Wu, J.-W. Su, Y.-L. Chung, L.-Y. Hong, J.-S. Ren, F.-C. Chang, Y. Wu, H.-Y. Chen, C.-H. Lin, H.-M. Hsiao *et al.*, "A 28nm 1mb time-domain computing-in-memory 6t-sram macro with a 6.6 ns latency, 1241gops and 37.01 tops/w for 8b-mac operations for edge-ai devices," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 1–3.
- [8] H. Fujiwara, H. Mori, W.-C. Zhao, M.-C. Chuang, R. Naous, C.-K. Chuang, T. Hashizume, D. Sun, C.-F. Lee, K. Akarvardar *et al.*, "A 5-nm 254-tops/w 221-tops/mm² fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous mac and write operations," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 1–3.
- [9] D. Wang, C.-T. Lin, G. K. Chen, P. Knag, R. K. Krishnamurthy, and M. Seok, "Dimc: 2219tops/w 2569f2/b digital in-memory computing macro in 28nm based on approximate arithmetic hardware," in *2022 IEEE international solid-state circuits conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 266–268.
- [10] Y.-D. Chih, P.-H. Lee, H. Fujiwara, Y.-C. Shih, C.-F. Lee, R. Naous, Y.-L. Chen, C.-P. Lo, C.-H. Lu, H. Mori *et al.*, "16.4 an 89tops/w and 16.3 tops/mm² all-digital sram-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64. IEEE, 2021, pp. 252–254.
- [11] K. Yoshioka, "A 818–4094 tops/w capacitor-reconfigured analog cim for unified acceleration of cnns and transformers," *IEEE Journal of Solid-State Circuits*, 2024.
- [12] H. Zhu, H. Zhang, S. He, M. Li, X. Zeng, and C. Chen, "Trident-cim: A lut-based compute-in-memory macro with trident read bit-line and partial product pruning," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 5, pp. 2544–2548, 2024.
- [13] C.-T. Lin, D. Wang, B. Zhang, G. K. Chen, P. C. Knag, R. K. Krishnamurthy, and M. Seok, "Dimca: An area-efficient digital in-memory computing macro featuring approximate arithmetic hardware in 28 nm," *IEEE Journal of Solid-State Circuits*, vol. 59, no. 3, pp. 960–971, 2023.
- [14] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.
- [15] D. Kalamkar, D. Mudigere, N. Mellemudi, D. Das, K. Banerjee, S. Avancha, D. T. Vooturi, N. Jammalamadaka, J. Huang, H. Yuen *et al.*, "A study of bfloat16 for deep learning training," *arXiv preprint arXiv:1905.12322*, 2019.
- [16] Y. Mai, M. Wang, C. Zhang, B. Zhong, and Z. Yu, "A 1.97 tflops/w configurable sram-based floating-point computation-in-memory macro for energy-efficient ai chips," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2023, pp. 1–5.
- [17] M. Li, H. Zhang, S. He, H. Zhu, H. Zhang, J. Liu, J. Chen, Z. Hu, X. Zeng, and C. Chen, "A 19.7 tflops/w multiply-less logarithmic floating-point cim architecture with error-reduced compensated approximate adder," in *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2024, pp. 1–5.
- [18] F. Tu, Y. Wang, Z. Wu, L. Liang, Y. Ding, B. Kim, L. Liu, S. Wei, Y. Xie, and S. Yin, "A 28nm 29.2 tflops/w bf16 and 36.5 tops/w int8 reconfigurable digital cim processor with unified fp/int pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 1–3.
- [19] J. Lee, J. Kim, W. Jo, S. Kim, S. Kim, J. Lee, and H.-J. Yoo, "A 13.7 tflops/w floating-point dnn processor using heterogeneous computing architecture with exponent-computing-in-memory," in *2021 Symposium on VLSI Circuits*. IEEE, 2021, pp. 1–2.
- [20] S. Jeong, J. Park, and D. Jeon, "A 28nm 1.644 tflops/w floating-point computation sram macro with variable precision for deep neural network inference and training," in *ESSCIRC 2022-IEEE 48th European Solid State Circuits Conference (ESSCIRC)*. IEEE, 2022, pp. 145–148.
- [21] C. Zhang, M. Wang, Y. Mai, C. Tang, and Z. Yu, "A high-density and reconfigurable sram-based digital compute-in-memory macro for low-power ai chips," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, no. 9, pp. 3589–3593, 2023.
- [22] A. Guo, C. Xi, F. Dong, X. Pu, D. Li, J. Zhang, X. Dong, H. Gao, Y. Zhang, B. Wang *et al.*, "A 28-nm 64-kb 31.6-tflops/w digital-domain floating-point-computing-unit and double-bit 6t-sram computing-in-memory macro for floating-point cnns," *IEEE Journal of Solid-State Circuits*, vol. 59, no. 9, pp. 3032–3044, 2024.
- [23] M. Li, H. Zhu, S. He, H. Zhang, J. Liao, D. Zhai, C. Chen, Q. Liu, X. Zeng, N. Sun *et al.*, "Slam-cim: A visual slam backend processor with dynamic-range-driven-skipping linear-solving fp-cim macros," *IEEE Journal of Solid-State Circuits*, vol. 59, no. 11, pp. 3853–3865, 2024.